Cairo University
Faculty of Computers & Artificial Intelligence
Department of Computer Science
Artificial Intelligence Course

Lab 1: Introduction to Prolog

Artificial intelligence (AI) aims at creating intelligent systems that can perform tasks, solve problems, and make decisions the way humans do. AI has various subfields which include knowledge representation, reasoning, planning, search, machine learning, natural language processing, computer vision, robotics, etc.

Knowledge representation is the subfield of AI dedicated to representing information about the world in a form that a computer system can use to solve complex tasks. Since we will focus on knowledge representation, we should start with a language of some sort, in terms of which knowledge can be formulated. We will use a very popular **logic programming language** called Prolog that has its roots in first-order logic (FOL).

Prolog Overview:

Prolog (**Pro**gramming in **Log**ic) is a **declarative** programming language. Unlike imperative programming languages (e.g. C or Java) in which we specify how to achieve a certain goal in a certain situation, in Prolog we specify the situation (rules and facts) and the goal (query/question), and Prolog derives the solution from the given knowledge base.

SWI-Prolog:

Download page: https://www.swi-prolog.org/Download.html

- SWI-Prolog is a Prolog **programming environment** in which we load program files and run queries.
- The Prolog code is written in a **text-based** file (or more than one file) using a standard text editor.
- These files are saved with a **.pl** extension.
- SWI-Prolog is then instructed to read the programs from these files (which is called **consulting** the files).

Consulting a program file in Prolog:

- Open SWI-Prolog, click **File** → **Consult** → **Browse for (.pl files)**
- Another way to input a program from a file is to use the query: `?- consult ('C:\myfile.pl').`
- The consult query can also take multiple file names as argument: `?- consult ([file1, file2, file3]).`
- If your file is already loaded in Prolog but you made some edits and want Prolog to see the new file then you can use: `?- make.`

Writing Programs in Prolog:

A Prolog program is a **knowledge base** (KB) containing **facts and rules** that define objects in a domain and the relations between them.

Types of clauses in Prolog:

- **Facts:** define things that are unconditionally (always) true.
- **Rules:** define things that are true depending on given conditions.
- **Queries:** are questions or goals that the Prolog interpreter tries to prove.

*Note: Each clause or goal must **end with a period (.)** in Prolog. You can write a clause in as many lines as you want, using a period to terminate the entire clause.*

For example, we can create a **.pl** file and write the following facts in it:

```
apples_are_fruits.
fruit(apples).
```

Notice that the facts above are defined in different ways, but they are both still facts in the knowledge base, and to us, they mean the same thing. To use your knowledge base in Prolog, save the file and load it in SWI-Prolog using **"consult".**

Now let's try writing some queries in SWI-Prolog:

```
?- apples_are_fruits.
true.
?- fruit(apples).
true.
```

```
?- fruits_include_apples.
ERROR.
?- fruit(grapes).
false.
```

Let's understand what happened in the previous queries:

- When we enter a query, Prolog considers it a goal and tries to prove it using the facts and rules defined in the knowledge base.
- So, in the first example, the query we entered exists as it is as a fact in the knowledge base, so Prolog can easily prove it and it prints **"true".** The same thing happened with the second example.
- In the third example, we entered a statement that has the same meaning as the first two statements. However, unlike humans, Prolog doesn't understand this, and it doesn't recognize or find this goal in (or infer it from) the knowledge base. That is why it shows an error message.
- In the last example, Prolog recognizes that **"fruit()"** exists in the knowledge base, however it is only true when its argument is **"apples"** (otherwise it will be false). That is why Prolog prints **"false"** not an error in this example.

Before we get into the details and examples of facts, rules, and queries, we need to understand the building blocks of these clauses, i.e. Prolog data objects and operators.

Data Objects:

- **Constants (atoms and numbers):**

  They are made up of the following alphabet:

  - Letters from A to Z and a to z
  - Digits from 0 to 9
  - Special characters such as _

  Atoms need to start with a **lowercase letter.** Strings of characters enclosed in single quotes are also atoms. This makes it possible to have an atom that starts with a capital letter.

  Numbers can be integers or real numbers.

Here are some examples of constants:

```
car.
apples_are_fruits.
"The sun rises from the east".
```

- **Variables:**

  Variables are symbol expressions beginning with an **uppercase letter,** e.g. X, Y, Z, Building.

  Variables can be used in facts and rules to make them generic, and they can be used in queries to manipulate Prolog into returning actual answers/values instead of just "true" and "false".

  Anonymous variables: When a variable appears in a clause only once, we don't have to give it a name, we can just use underscore instead. Each time an underscore occurs in a clause it represents a new anonymous variable.

- **Structures:**

  They are objects (predicates) that have one or more components. They start with a **lowercase letter,** and usually describe relations between components. The components themselves can in turn be structures!

  The number of components (arguments) in the predicate is called the arity.

  Here are some examples of structures in the knowledge base:

```
odd_num(3). % A predicate called "odd_num" that is true
% only if its argument is 3 as it is defined only with 3
% in the KB.

thing(X).  % A predicate called "thing" that is true for
% any argument as it is defined with a variable in the KB.

fruit(apples).
fruit(apples, grapes, oranges).
% Although these two predicates have the same name, they
% are different because their arity is different.

friend(nour, nada).  % A relation called "friend" that can
% be read as "Nour is a friend of Nada".

color(car(amr)).  % A predicate of a predicate of an atom.
```

- **Lists** *(will be discussed in the third lab)*

Operators:

In Prolog, we have the common arithmetic and comparison operators in addition to some special operators. However, the operators should be used in a certain way such that they form clauses.

Consider the following query:

```
?- 2 + 3.
ERROR.
```

Prolog doesn't consider this an expression to be evaluated! Prolog considers it a goal to be proven and fails to find it. So, we can manipulate Prolog into actually evaluating the expression by using variables.

```
?- X is 2 + 3.
X = 5.
```

Prolog still considers the previous query a goal to be proven. However, to prove that it is true, ***X*** needs to be equal to ***5***, which explains why Prolog printed **X=5** (as it prints the value of the variable which makes the goal true).

Notice that we used the operator **"is"** because it is the operator that performs the arithmetic operations on the right hand side (unlike **"="** which doesn't perform arithmetic operations).

Here are some examples of using operators:

```
?- 2 = 2.
true.
?- 3 = 2 + 1.
false.
?- 3 is 2 + 1.
true.
?- X is (3 + 1) / 2 * 8.
X = 16.
% We can get the remainder of the division using "mod" operator:
?- X is 9 mod 2.
X = 1.
```

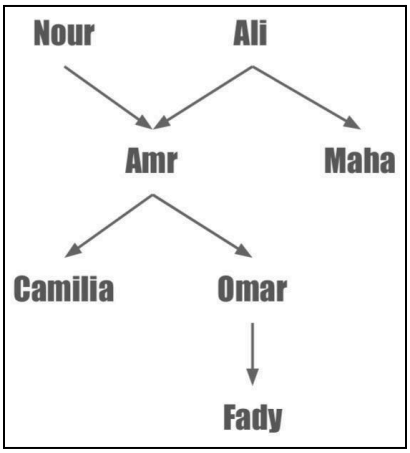
```
% Integer division:
?- X is 9 // 2.
X = 4.

% "is" only evaluates what is on its right.
?- 3 + 1 is 2 + 2.
false.

% =:= evaluates expressions both on its right and left.
?- 3 + 1 =:= 2 + 2.
true.

% \= checks that the terms on its sides can't be unified.
?- 4 \= 4.
false.

?- 4 \= 2+2.
true.

% =\= evaluates both expressions on its right and its left.
?- 4 =\= 2+2.
false.

?- X = 2.
X = 2.

% == returns true if its operands are identical without unifying them.
?- X == 2.
false.

% \== returns true if terms aren't identical even if they can be unified.
?- X \== 2.
true.

?- X \= 2.
false.
```

Notice the difference between =, is and =:=, the difference between = and ==, and the difference between \= and \==.

We will now use the data objects and operators we learnt to write facts, rules and queries.

Facts:

Suppose we have the following family tree:



We would like to represent this family tree in a knowledge base by defining the predicates **"male"** and **"female",** and the **"parent"** relation where **"parent(X, Y)"** can be read as "X is the parent of Y".

So, we can create a **.pl** file and write the following facts in it:

```
male(ali).
male(amr).
male(omar).
male(fady).

female(nour).
female(maha).
female(camilia).

parent(nour, amr).
parent(ali, amr).
parent(ali, maha).
parent(amr, camilia).
parent(amr, omar).
parent(omar, fady).
```

Each clause in the last part declares one fact about the parent relation. The first clause for example is a particular instance of the parent relation. Such an instance is also called a **relationship.**

A question can be:

```
parent(ali, amr).
```

```
true.
```

```
parent(amr, X).
```

X = camilia;

X = omar.

Rules:

In Prolog, rules consist of:

- **Head:** conclusion on the left hand side of the rule that is true based on certain conditions.
- **Body:** conditions on the right hand side of the rule.

The conditions in the body can be goals separated by commas (conjunction) and sometimes semicolon (disjunction).

*Note: Prolog rules of the same name form **procedures.***

Suppose we want to define the **"offspring"** relation where:

For all X and Y

Y is an offspring of X if

X is a parent of Y

The corresponding Prolog rule which has the same meaning is:

```
offspring(Y, X):-  % ":-" means if.
     parent(X, Y).
```

Let's define more rules representing some of the other relationships, and put them in our knowledge base.

```
grandparent(X, Y):-
     parent(X, Z),
     parent(Z, Y).

father(X, Y):-
     male(X),
     parent(X, Y).

sister(X, Y):-
     female(X),
     parent(Z, X),
     parent(Z, Y),
     X \= Y.

aunt(X, Y):-
     parent(Z, Y),
     sister(X, Z).
```

*Note: The **lexical scope** of variable names is one clause. This means that if the name X occurs in two clauses, then it signifies two different variables. However, the occurrence of X within the same clause means the same variable.*

What if we wanted to define a **"predecessor"** relation?

This relation can be defined in terms of the parent relation. The whole definition can be expressed with two rules. The first rule will define the direct predecessor and the second rule for the indirect predecessors. We say that X is an indirect predecessor of Y if there is a chain of parent relationships between X and Y.

So, the definition of the predecessor can be as follows:

For all X and Y,

X is a predecessor of Y if

X is the parent of Y

***or** if*

there is a Z such that

X is a parent of Z and

Z is a predecessor of Y.

The corresponding Prolog rule which has the same meaning is:

```
predecessor(X, Y):-
     parent(X, Y).

predecessor(X, Y):-
     parent(X, Z),
     predecessor(Z, Y).
```

Notice that this is a **recursive rule**! The key idea of the recursive rule is that we can define a rule that calls itself. The recursive calls should stop when a base case (base rule) is reached. Recursive programming is in fact one of the fundamental principles in Prolog programming, and this will be the topic of the next lab.

Queries:

Let's consult our family tree knowledge base and ask Prolog some questions and observe the outputs:

```
?- parent(X, Y), X \= ali.
X = nour,
Y = amr;
X = amr,
Y = camilia;
X = amr,
Y = omar;
X = omar,
Y = fady.
?- grandparent(X,fady).
X = amr;
false.
?- father(ali, fady).
false.
?- sister(X, Y).
X = maha,
Y = amr;
X = camilia,
Y = omar.
?- aunt(X, _).
X = maha;
X = maha;
X = camilia.
?- predecessor(ali, fady).
true.
?- predecessor(X, fady).
X = omar;
X = nour;
X = ali;
X = amr;
false.
```

How Prolog answers our questions:

- A query/question is a sequence of one or more goals. To answer a question, Prolog tries to satisfy all the goals. "To satisfy a goal" means to demonstrate that the goal is true, assuming that the relations in the program are true.
- If the question contains variables, Prolog has to find the particular objects (in place of variables) for which the goal is satisfied. The particular instantiation of variables to these objects is displayed to the user.
- If Prolog cannot demonstrate for some instantiations of variables that the goals logically flow from the program, then the Prolog's answer to the question will be "false".
- If we enter the trace mode using the query "`?- trace.`", we will see that Prolog finds the proof sequence in the inverse order (i.e. in a goal driven manner). Prolog starts with the goal and, using rules, substitutes the current goals with new sub goals, until new goals happen to be simple facts. If some goal fails, Prolog ***backtracks*** to try another alternative way to derive the top goal until it either succeeds or fails to find an adequate substitution.

Exercise: Open the trace mode and observe the trace of the previous queries. You can close the trace mode using "`?- notrace.`"

Unification and Matching:

- **Unification** is the process of determining whether two expressions can be unified, i.e. made identical by appropriate substitution for their variables.
- **Substitution** is a finite set of pairs of variables and terms called replacements.

Given two terms, we say that they match if:

- They are identical, or
- The variables in both terms can be instantiated to objects in such a way that after the substitution of variables by these objects the terms become unified/identical.

The general rules for matching two terms T1 and T2 are:

- If T1 and T2 are constants then they match only if they are the same thing.
- If T1 is a variable and T2 is anything then they match (T1 is bound to T2).
- If T1 and T2 are structures then they match only when they have the same principal functor, the same arity, and all their corresponding components match.

Backtracking:

Consider the following program:

```
hold_party(X):-
     birthday(X),
     happy(X).

birthday(amr).
birthday(ali).
birthday(nada).

happy(maha).
happy(mai).
happy(nada).
```

If we pose the query:

```
?- hold_party(X).
```

What happens is:

- Prolog tries to bind X to a constant through **birthday(X)**, i.e., **birthday(X)** becomes "subgoal1".
- **birthday(X)** matches with **birthday(amr)** in the KB, so **X = amr**, and subgoal1 is true.
- Now it's time to prove **happy(amr)** as "subgoal2". Prolog traverses the knowledge base to prove it but fails to find it.
- Prolog **backtracks** to the nearest possible subgoal that has an alternative path. So, it tries to prove **birthday(X)** with some value other than "amr" (subgoal1').
- **birthday(X)** matches with **birthday(ali)**, so **X = ali** and subgoal1' is true.
- Now it is time to prove **happy(ali)**. Prolog traverses the knowledge base to prove it but fails.
- Prolog backtracks and tries to prove **birthday(X)** with some other value.
- **birthday(X)** matches with **birthday(nada)**, so **X = nada** and subgoal1'' is true.
- Now it is time to prove **happy(nada)**. Prolog traverses the knowledge base to prove it and succeeds!